

Subiect 07 – Sistem de transport urban

Cerințe obligatorii

1. Pattern-urile implementate trebuie sa respecte definiția din GoF discutată în cadrul cursurilor și laboratoarelor. Nu sunt acceptate variații sau implementări incomplete.
2. Pattern-ul trebuie implementat în totalitate corect pentru a fi luat în calcul.
3. Soluția nu conține erori de compilare.
4. Pattern-urile pot fi tratate distinct sau pot fi implementate pe același set de clase.
5. Implementările care nu au legătura funcțională cu cerințele din subiect NU vor fi luate în calcul (preluare unui exemplu din alte surse nu va fi punctată).
6. NU este permisă modificare claselor primite.
7. Soluțiile vor fi verificate încrucișat folosind MOSS. Nu este permisă partajarea de cod între studenți. Soluțiile care au un grad de similitudine mai mare de 30% vor fi anulate.

Cerințe Clean Code obligatorii (soluția este depunctată cu câte 2 puncte pentru fiecare cerință ce nu este respectată) - maxim se pot pierde 8 puncte

1. Pentru denumirea claselor, funcțiilor, testelor unitare, atributelor și a variabilelor se respecta convenția de nume de tip Java Mix CamelCase;
2. Pattern-urile și clasa ce conține metoda main() sunt definite in pachete distincte ce au forma *cts.nume.prenume.gNrGrupa.denumire_pattern*, *cts.nume.prenume.gNrGrupa.main* (studenții din anul suplimentar trec "as" în loc de gNrGrupa);
3. Clasele și metodele sunt implementate respectând principiile KISS, DRY și SOLID (atenție la DIP);
4. Denumirile de clase, metode, variabile, precum și mesajele afișate la consola trebuie sa aibă legătura cu subiectul primit (nu sunt acceptate denumiri generice). Funcțional, metodele vor afișa mesaje la consola care sa simuleze acțiunea cerută sau vor implementa prelucrări simple.

Se dezvoltă o aplicație software destinată unui sistem de transport urban

3p. O aplicație de transport urban trebuie să calculeze ruta optimă între două stații. Utilizatorul poate alege criteriul de optimizare: cel mai scurt timp, cel mai mic număr de stații, cost minim sau traseu cu cele mai puține schimbări. Fiecare variantă folosește o regulă proprie de calcul asupra aceleiași liste de stații și conexiuni. Algoritmul utilizat trebuie să poată fi schimbat la rulare. Pentru implementare se va folosi interfața *AbstractAlgoritmRuta*.

1.5p. Să se testeze soluția prin:

- definirea unei rețele simple de transport;
- implementarea a cel puțin 3 variante de calcul;
- calcularea rutei pentru aceeași pereche de stații folosind algoritmi diferiți;
- compararea rezultatelor obținute.

3p. În aplicație, stațiile și grupurile de stații trebuie reprezentate uniform. O stație simplă are un număr zilnic de pasageri, iar un nod de transport poate conține stații simple, peroane sau alte noduri. Se dorește calcularea recursivă a numărului total de pasageri și validarea faptului că un nod nu depășește capacitatea maximă admisă. Pentru implementare se va folosi interfața *AbstractElementTransport*.

1.5p. Să se testeze soluția prin:

- crearea unor stații simple;
- agregarea lor în noduri de transport;
- calcularea numărului total de pasageri;
- verificarea capacității pentru un nod complex.

```
public interface AbstractAlgoritmRuta {  
    Ruta calculeazaRuta(ReteaTransport retea, String statieStart, String  
    statieFinal);  
}
```

```
public interface AbstractElementTransport {  
    int calculeazaNumarPasageri();  
    boolean verificaCapacitate(int capacitateMaxima);  
}
```